



East West University

Department of Computer Science & Engineering

Lab Project Report

Course Title & Code: Structured Programming (CSE103)

Submitted To:

Khondaker Mohammad Mohi Uddin

Assistant Professor

Department of CSE

East West University

Submitted By:

Adnan Khondakar

Student ID: 2026-2-60-453

Aritro Huda

Student ID: 2026-2-60-455

Rashed Hasan

Student ID: 2026-2-60-466

Section: 16

Semester: Summer-26

Project: R.P.G Game

"Echoes of the Old Tower"

R.P.G GAME

Introduction

The R.P.G Game is a console-based role-playing game developed in the C programming language. The project combines structured programming concepts with an interactive game system in which the player can explore a free world, participate in battles, collect equipment, encounter characters, and complete a story-driven adventure.

The program is built around structures, arrays, functions, pointers, loops, conditional statements, random-number generation, string handling, and file operations. It provides two major gameplay modes: **Free World Mode** and **Story Mode**.

The game uses a reusable combat engine, a quiz-based boss battle system, random encounters, inventory resources, and a text-file save/load mechanism for Story Mode. The source code also separates common helpers, combat, quiz logic, free world logic, story logic, and the main menu into organized sections.

Objectives

The main objectives of this project are:

- To develop an interactive RPG-style game using the C programming language.
 - To demonstrate the practical use of structures and user-defined data types.
 - To use functions to divide a large program into manageable modules.
 - To implement pointers for modifying player and game-state data.
 - To apply arrays for storing quiz questions and other repeated game data.
 - To use loops and conditional statements for game flow and combat.
 - To implement random events and random enemy encounters.
 - To demonstrate file handling through Story Mode save/load functionality.
 - To provide an interactive menu-driven console interface.
-

Project Scope

The project focuses on demonstrating structured programming techniques rather than graphics programming. All interaction takes place through the terminal/console. The design nevertheless includes multiple game systems so that several C concepts can be demonstrated in one complete application.

System Features

The R.P.G Game includes the following major features.

1. Free World Mode

- Player creates a hero and starts with 400 HP.
- Player can roam a ruined road and later enter a desert zone.
- Abandoned buildings may contain multiple floors.
- Random enemy encounters occur while exploring.
- Items such as sword, knife, gun, grenades, healing potions and machine gun uses can be collected.
- A pet event may introduce a cat or dog companion.
- A one-time romance event introduces a companion character.
- Enemy difficulty progresses through B-Class, A-Class and S-Class enemies.
- Black Hulk encounters use the quiz battle system.
- Death restarts Free World Mode from the beginning.

2. Story Mode

- Supports new-player registration and returning-player login.
 - Stores player progress in **story_save.txt**.
 - Contains an introductory story and a tutorial fight against Dwyen.
 - Allows the player to select a loadout before entering the Old Tower.
 - Contains a Ground Floor checkpoint and multiple tower floors.
 - Includes hidden item pickups such as a gun and healing potions.
 - Contains a scripted Mini Hulk event and a comeback power-up.
 - The final Black Hulk encounter is a seven-question quiz battle.
 - Successful completion displays the story ending and removes the save file.
-

3. Combat System

The shared **fight()** routine handles ordinary battles. It supports multiple attacks, resource checks, enemy retaliation, missed attacks, damage calculation, healing, and victory/defeat detection. The same engine is reused across several parts of the game.

4. Quiz Battle System

Boss quiz battles contain three fixed questions and four randomly selected questions. A correct answer damages the boss, while a wrong answer causes the boss to attack the player.

Technical Implementation

1. Data Structures

The program uses several structures to organize related information.

Structure	Purpose	Important Members
Player	Stores the complete state of the hero.	name, hp, max_hp, strength_mult, weapon_mult, weapons, ammunition, potions, pet and companion information
Enemy	Represents an enemy template used by the combat system.	name, hp, damage, miss_chance, retaliate_every
QuizQuestion	Stores one multiple-choice question.	text, option1, option2, option3, correct
StoryProgress	Tracks Story Mode checkpoint and floor progress.	at_checkpoint, floor1_cleared, floor2_cleared, comeback_done, item availability flags

Player Structure

```
typedef struct {
    char name[MAX_NAME];
    int hp;
    int max_hp;
    int strength_mult;
    int weapon_mult;

    int has_knife;
    int has_sword;
    int gun_shots;
    int grenades;
    int healing_potions;
    int machine_gun_uses;

    int has_pet;
    char pet_name[MAX_NAME];

    char wife_name[MAX_NAME];
    int has_companion;
    char companion_name[MAX_NAME];
} Player;
```

Enemy Structure

```
typedef struct {
    char name[MAX_NAME];
    int hp;
    int damage;
    int miss_chance;
    int retaliate_every;
} Enemy;
```

Arrays and Constants

1. Program Constants

Constant	Value	Purpose
MAX_NAME	50	Maximum stored length for names.
MAX_LOGIN	40	Maximum size for user ID and password fields.
QUIZ_POOL	20	Total questions in the random quiz pool.
QUIZ_FIXED	3	Number of questions always asked.
QUIZ_RANDOM	4	Number of random questions selected.
SAVE_FILE	story_save.txt	Text file used for Story Mode persistence.

2. Quiz Question Arrays

The program uses one fixed-question array and one random question pool. The fixed array contains three questions that are always presented in order. The random pool contains twenty questions, from which four distinct questions are selected for each quiz battle.

```
static QuizQuestion fixed_questions[QUIZ_FIXED] = {
    { "Who got to the moon for the very first time?",
      "Neil Armstrong", "Adnan Kondakar", "Rashed Hasan", 1 },
    { "What is the name of the highest mountain in the world?",
      "Aritro the Peak", "Mount Everest", "Tajingdong", 2 },
    { "What is the favourite word of K M M U sir?",
      "Ghorar Dim", "Hunky Punky", "Bondhu", 3 }
};
```

3. Random Selection

The function **pick_random_questions()** generates distinct random indexes. It repeatedly generates an index and checks whether it was already selected. This prevents the same random question from being selected twice in one battle.

```
candidate = rand() % QUIZ_POOL;

for (j = 0; j < i; j++)
    if (chosen[j] == candidate)
        already_used = 1;
```

Functions and Program Modules

The program is organized into reusable functions rather than putting all logic inside **main()**.

Function	Role
pause_game()	Pauses the game until the user presses Enter.
read_int()	Safely reads an integer inside a specified range.
read_line()	Reads a non-empty text line.
heal_player()	Restores HP without exceeding maximum HP.
print_player()	Displays player health and available equipment.
player_turn()	Displays combat actions and calculates the selected action.
fight()	Shared ordinary combat engine.
quiz_battle()	Runs the multiple-choice boss battle.
run_free_world()	Controls Free World Mode.
run_story_mode()	Controls Story Mode and save/load flow.
save_game()	Writes Story Mode state to a text file.
load_game()	Reads and validates saved Story Mode state.
st_ground_floor()	Runs the Old Tower ground floor.
st_first_floor()	Runs the first floor and hidden gun event.
st_second_floor()	Runs the second floor, potion event and Mini Hulk.
st_comeback()	Applies the story power-up after the first Mini Hulk encounter.
st_ending()	Displays the successful Story Mode ending.

Modular Design

This modular structure improves readability and makes it possible to reuse common functionality. For example, **fight()** is shared by Free World encounters and Story Mode fights, while **quiz_battle()** is reused for the S-Class Black Hulk encounters.

Methodology

1. Program Flow

1. **Program Start:** `main()` initializes random number generation and displays the game title.
2. **Main Menu:** The player chooses Free World Mode, Story Mode, or Exit.
3. **User Input:** Input is read through validation functions such as `read_int()` and `read_line()`.
4. **Mode Execution:** The selected mode calls its corresponding control function.
5. **Game Processing:** Encounters, battles, items, movement, story events and quizzes are processed.
6. **State Update:** Player HP, weapons, ammunition, progress flags and other fields are updated.
7. **Loop Back:** The game continues until the player exits, loses, or completes the story.

main() → Main Menu → Free World / Story Mode → Events & Combat → State Update → Continue / Exit

2. Key Programming Concepts Used

- **Structures:** Used to group player, enemy, quiz and story progress information.
- **Arrays:** Used for question banks and fixed enemy/question data.
- **Pointers:** Used to modify structures through function parameters.
- **Functions:** Used to divide the program into logical modules.
- **Loops:** Used for game loops, combat rounds and repeated input.
- **Conditional Statements:** Used for branching game events and decisions.
- **Strings:** Managed using `fgets()`, `strncpy()`, `strcmp()` and related operations.
- **File Handling:** Used to save and restore Story Mode progress.
- **Randomization:** Used for encounters, enemy selection, zones and quiz questions.

3. Input Validation

The `read_int()` function repeatedly asks for input until the value is within the required range. The function also rejects trailing non-numeric characters. This reduces invalid menu choices and improves the reliability of the console interface.

Combat System

Free World Combat

Free World Mode supports the following actions:

Action	Base Effect	Resource
Punch in the belly	5 damage	None
Punch in the face	10 damage	None
Kick in the belly	20 damage	None
Knife	12 damage	Knife
Sword	50 damage	Sword
Gun	55 damage	1 shot
Grenade	100 damage	1 grenade
Machine gun	200 damage	1 use
Healing potion	+50 HP	1 potion

Story Mode Combat

Story Mode uses a smaller move list: sword, gun, grenade and healing potion. Weapon damage is multiplied by **weapon_mult**, while unarmed damage in Free World Mode uses **strength_mult**.

Enemy Retaliation

After a successful player attack, the combat engine counts the number of landed hits. The enemy can retaliate according to **retaliate_every**. An enemy may also have a non-zero **miss_chance**, allowing attacks to miss.

```
enemy.hp -= result;
if (enemy.hp < 0) enemy.hp = 0;

landed_hits++;

if (enemy.hp <= 0) break;

/* enemy retaliation */
if (landed_hits % n == 0) {
    if (enemy.miss_chance > 0 &&
        (rand() % 100) < enemy.miss_chance) {
        printf("%s attacks but misses!\n", enemy.name);
    } else {
        p->hp -= enemy.damage;
    }
}
```

The combat function receives an enemy template by value and creates a local copy. Therefore, damage taken during one battle does not permanently modify the original enemy template.

Free World Mode

Exploration

Free World Mode starts the player at 400 HP. The player can travel along a ruined road, enter abandoned buildings, and eventually reach a desert area. The zone changes after enough road travel.

Random Events

- Romance event.
- Random companion dialogue after the romance event.
- Item pickup.
- Pet event.
- Random enemy encounter.
- Black Hulk quiz battle after lower enemy tiers are cleared.

Enemy Progression

Class	Enemies	Purpose
B-Class	Goon, Thief, Wild Dog	Early encounters.
A-Class	Robot, Cyborg, Mutant Dinosaur	Stronger mid-game encounters.
S-Class	Black Hulk	High-level quiz boss.

Item Pickup

When an item event occurs, the player may choose one of several items or leave the box. Weapons affect combat, while some flavor items do not change player statistics.

```
printf(" 1) Stick\n 2) Sword\n 3) Knife\n 4) Gun (20 shots)\n");
printf(" 5) Torch\n 6) Healing potion\n 7) Grenade\n");
printf(" 8) Machine gun (2 uses)\n 9) Leave it\n");
```

Story Mode

Story Introduction

Story Mode begins with a login screen. A new player chooses a user ID, password, hero name and wife's name. The introductory narrative then establishes the kidnapping storyline and leads the player toward Detective Holmes and the Old Tower.

Login and Save System

Returning players provide the same user ID and password used when the game was saved. The program validates these values before loading the saved state.

```
loaded = load_game(user, pass, &p, &prog);

if (loaded == 0) {
    printf("\nNo save file found. Start a new game first.\n");
    return;
}
if (loaded == -1) {
    printf("\nIncorrect user ID or password.\n");
    return;
}
```

Old Tower Progression

1. **Ground Floor:** Three rooms with enemies. Reaching the stairs creates the checkpoint.
2. **First Floor:** The player chooses east or west and may find a hidden gun.
3. **Second Floor:** The player can find healing potions, fight the Mutant Dragon and encounter the Mini Hulk.
4. **Comeback:** After the scripted first loss to the Mini Hulk, Holmes provides a powerful upgrade.
5. **Final Floor:** The player faces Black Hulk in the quiz battle.

Login → Introduction → Dwyen → Loadout → Ground Floor Checkpoint → 1st Floor → 2nd Floor → Comeback → Final Boss → Ending

Save and Load Mechanism

Saving Game Data

Story Mode writes player and progress data to a plain-text file named **story_save.txt**. The program writes one logical group of values per line, including login information, character information, combat statistics, inventory and story progress.

```
static int save_game(const char *user, const char *pass,
                    const Player *p,
                    const StoryProgress *prog)
{
    FILE *fp = fopen(SAVE_FILE, "w");
    if (!fp) {
        printf("Could not write save file.\n");
        return 0;
    }

    fprintf(fp, "%s\n%s\n", user, pass);
    fprintf(fp, "%s\n%s\n", p->name, p->wife_name);
    fprintf(fp, "%d %d %d %d\n",
            p->hp, p->max_hp,
            p->strength_mult, p->weapon_mult);

    fclose(fp);
    return 1;
}
```

Loading Game Data

load_game() opens the save file, reads the stored credentials, compares them with the supplied credentials, and then restores the player and progress fields. The program returns different status values for success, missing save file and incorrect credentials.

StoryProgress Structure

Field	Meaning
at_checkpoint	Whether the ground-floor checkpoint has been reached.
floor1_cleared	Whether the first floor has been completed.
floor2_cleared	Whether the second floor has been completed.
comeback_done	Whether the Holmes power-up has been received.
gun_pickup_available	Whether the hidden gun can still be collected.
potion_pickup_available	Whether the hidden potions can still be collected.

The save system is intentionally simple and educational. It uses a plain text file rather than a database, making it appropriate for a structured programming course project.

Quiz Battle System

Boss Quiz Design

The quiz engine is shared by Free World S-Class Black Hulk encounters and the Story Mode final boss. Each battle consists of three fixed questions followed by four randomly selected questions, for a maximum of seven questions.

Scoring Logic

The damage for each correct answer is calculated from the boss's HP and the total number of questions. This means a perfect seven-answer run is sufficient to defeat the boss regardless of the boss's configured HP.

```
int per_correct_damage =  
    (boss.hp + (QUIZ_FIXED + QUIZ_RANDOM - 1))  
    / (QUIZ_FIXED + QUIZ_RANDOM);
```

Question Processing

1. Display the question text.
2. Display three answer options.
3. Read an answer from 1 to 3.
4. If correct, subtract damage from the boss.
5. If incorrect, subtract the boss's damage from the player.
6. Continue until all questions are processed or one side is defeated.

Example Question Categories

- C programming expressions and operators.
- Loops and conditional statements.
- String functions.
- General knowledge.
- Science and geography.

Program Operations

1. Starting the Game

The **main()** function seeds the random-number generator using the current time and displays the title **Echoes of the Old Tower**. It then repeatedly shows the main menu until the player chooses Exit.

```
srand((unsigned int) time(NULL));

while (1) {
    printf("\nMAIN MENU\n");
    printf(" 1) Free World Mode\n");
    printf(" 2) Story Mode\n");
    printf(" 3) Exit\n");

    choice = read_int("> ", 1, 3);

    if (choice == 1) run_free_world();
    else if (choice == 2) run_story_mode();
    else break;
}
```

2. Player Healing

The **heal_player()** function adds the requested amount of HP but prevents the player's HP from exceeding the maximum HP.

3. Player Status

The **print_player()** function uses a pointer to access player data and is declared with **const Player *p** because it only reads the player's state and should not modify it.

```
void print_player(const Player *p)
{
    printf("%-18s %d/%d hp\n",
           p->name, p->hp, p->max_hp);

    if (p->has_knife) printf("Knife\n");
    if (p->has_sword) printf("Sword\n");
    if (p->gun_shots > 0)
        printf("Gun shots left: %d\n", p->gun_shots);
}
```

This is an example of const-correctness: the function can inspect the Player object through the pointer, but it cannot legally change the fields through p.

Testing and Output Screenshots

The terminal screens below demonstrate the primary functions and gameplay loops of the project, covering both Free World Mode and Story Mode.

```
=====
Echoes of the Old Tower
=====

MAIN MENU
1) Free World Mode
2) Story Mode
3) Exit
> 1

=====
FREE WORLD MODE
Here you roam and see the ruins of the existing world
=====
Enter your hero's name: Kyle
Starting hp: 400

You are traveling along the ruined road. (Distance: 0)
1) Roam the road
2) Enter nearby building
3) Quit to main menu
> 1

You roam further down the highway...

(press ENTER to continue)

You are traveling along the ruined road. (Distance: 1)
1) Roam the road
2) Enter nearby building
3) Quit to main menu
> 2

You enter an abandoned building with 4 floor(s).

--- Building Floor 1 ---

Amid the ruins, someone catches your eye -- and she is none other than your love, who is fighting to survive just like you..
Something inside you , all of a sudden , told you to confess your love towards her.. And you did it..And guess what , she blushed and said 'It took you that
long, you moron'... Now she is with you, what was her name again?? : Selina
Selina now roams the city with you. And you two make a wonderful couple
```

Figure 1: Main Menu, Free World Mode initialization, hero creation, exploration along the ruined road, and a random romance encounter.

```
Choose an action:
1) Punch in the belly (5 dmg)
2) Punch in the face (10 dmg)
3) Kick in the belly (20 dmg)
4) Knife (12 dmg) [not carried]
5) Sword (50 dmg) [not carried]
6) Gun (55 dmg, 17 shots left)
7) Grenade (100 dmg, 0 left)
8) Machine gun (200 dmg, 0 uses left)
9) Healing potion (+50 hp, 0 left)
10) View status
> 6
You deal 55 damage. Wild Dog hp: 0

You defeated the Wild Dog!

(press ENTER to continue)

--- Building Floor 3 ---

You spot a box of items on the ground. You can take only one... What will you take?
1) Stick
2) Sword
3) Knife
4) Gun (20 shots)
5) Torch
6) Healing potion
7) Grenade
8) Machine gun (2 uses)
9) Leave it
> 2
Sword acquired!

=====
A Goon appears! (100 HP)
=====

Your hp: 372/400 | Goon hp: 100

Choose an action:
1) Punch in the belly (5 dmg)
```

Figure 2: Combat resolution against a Wild Dog, the item-pickup event on Building Floor 3, and a subsequent Goon encounter.

Testing and Output Screenshots (Cont.)

```
=====
Echoes of the Old Tower
=====

MAIN MENU
1) Free World Mode
2) Story Mode
3) Exit
> 2

=====
STORY MODE LOGIN
=====
1) New player
2) Returning player
> 1
Choose a user ID: Moonnight
Choose a password: 3641992
Your hero's name: Sakib Al Hasan
Your wife's name: Shishir

=====
STORY MODE
=====

You are wealthy, successful, and a skilled martial artist. However, you sure have a lot of enemies
Tonight you are out to dinner with your lovely wife when a sudden explosion rocks the street outside.
She crumbles from fear
You want to go to check, but she stops you... You assure its ok and you will be back soon.

1) What happened next? 2) Skip to the game: 1

You go anyway... But that is the mistake you will forever remember
When you turn back, your wife is gone.

The kidnappers appear tied to enemies from your past business dealings.
The police can't help. You decide to find their warehouse yourself.

1) What happened next? 2) Skip to the game: 1

You tell your old friend Detective Holmes and spend a day searching for leads -- nothing.
```

Figure 3: Story Mode login screen, new-player registration, and the introductory kidnapping narrative.

```
2) Gun (85 dmg, 14 shots)
3) 5 grenades (200 dmg each)
4) 3 healing potions (+50 hp each)
Pick one: 3

1) Sword (70 dmg, unlimited uses) [chosen]
2) Gun (85 dmg, 14 shots)
3) 5 grenades (200 dmg each) [chosen]
4) 3 healing potions (+50 hp each)
Pick one: 4

Loadout locked in.

=====
GROUND FLOOR (checkpoint)
=====

Room 1: an enemy blocks your way. And you have to beat it to move forward

=====
A Big Dog appears! (60 HP)
=====

Your hp: 228/250 | Big Dog hp: 60

Choose an action:
1) Sword (70 dmg)
2) Gun (85 dmg, 0 shots left)
3) Grenade (200 dmg, 5 left)
4) Healing potion (+100 hp, 3 left)
5) View status
> 1
You deal 70 damage. Big Dog hp: 0

You defeated the Big Dog!
Exits: NORTH, EAST
1) East 2) North : 1

Room 2: an enemy blocks your way. And you have to beat it to move forward

=====
```

Figure 4: Pre-tower loadout selection and the Ground Floor checkpoint, including a Big Dog encounter.

Testing and Output Screenshots (Cont.)

```
Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 10 shots left)
3) Grenade (400 dmg, 5 left)
4) Healing potion (+100 hp, 4 left)
5) View status
> 3
You deal 400 damage. Mutant Dragon hp: 100
Mutant Dragon hits you for 180 damage! Your hp: 800/900
Your hp: 800/900 | Mutant Dragon hp: 100

Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 10 shots left)
3) Grenade (400 dmg, 4 left)
4) Healing potion (+100 hp, 4 left)
5) View status
> 2
You deal 170 damage. Mutant Dragon hp: 0
You defeated the Mutant Dragon!

Exit NORTH leads to the Stairs Room.
1) Proceed to Stairs Room
> 1

You reach the Stairs Room -- a massive shape rises in the dark. The Mini Hulk! His name can have mini in it
But surely he is not mini

=====
A Mini Hulk appears! (1000 HP)
=====

Your hp: 800/900 | Mini Hulk hp: 1000

Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 9 shots left)
3) Grenade (400 dmg, 4 left)
4) Healing potion (+100 hp, 4 left)
```

Figure 5: Defeat of the Mutant Dragon on the Second Floor, followed by the Mini Hulk boss encounter in the Stairs Room.

```
=====
FINAL FLOOR
=====

QUIZ BATTLE: Black Hulk (2000 HP)
3 fixed questions + 4 random questions.
=====

Who got to the moon for the very first time?
1) Neil Armstrong
2) Adnan Mondakar
3) Rashed Hasan
Answer: 1
Correct! You hit the Black Hulk for 286 damage. Boss hp: 1714

What is the name of the highest mountain in the world?
1) Aritro the Peak
2) Mount Everest
3) Tajingdong
Answer: 2
Correct! You hit the Black Hulk for 286 damage. Boss hp: 1428

What is the favourite word of K M M U sir?
1) Ghorar Dia
2) Hunky Punky
3) Bondhu
Answer: 3
Correct! You hit the Black Hulk for 286 damage. Boss hp: 1142

int a=10,b=3; printf("%d", a/b); Output?
1) 3
2) 3.33
3) 4
Answer: 1
Correct! You hit the Black Hulk for 286 damage. Boss hp: 856

int x=2; for(i=0;i<4;i++) x=x+3; Final x?
1) 11
2) 14
3) 15
```

Figure 6: The Final Floor quiz battle against Black Hulk, showing fixed and random questions being answered correctly.

Testing and Output Screenshots (Cont.)

```
> 4
You recover 100 hp. hp: 900/900
Your hp: 900/900 | Mini Hulk hp: 1000
Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 9 shots left)
3) Grenade (400 dmg, 4 left)
4) Healing potion (+100 hp, 3 left)
5) View status
> 3
You deal 400 damage. Mini Hulk hp: 600
Mini Hulk hits you for 200 damage! Your hp: 700/900
Your hp: 700/900 | Mini Hulk hp: 600
Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 9 shots left)
3) Grenade (400 dmg, 3 left)
4) Healing potion (+100 hp, 3 left)
5) View status
> 3
You deal 400 damage. Mini Hulk hp: 200
Mini Hulk hits you for 200 damage! Your hp: 500/900
Your hp: 500/900 | Mini Hulk hp: 200
Choose an action:
1) Sword (140 dmg)
2) Gun (170 dmg, 9 shots left)
3) Grenade (400 dmg, 2 left)
4) Healing potion (+100 hp, 3 left)
5) View status
> 3
You deal 400 damage. Mini Hulk hp: 0
You defeated the Mini Hulk!
The Mini Hulk falls at last.
```

Figure 7: The Mini Hulk defeated after a full-health engagement, clearing the way to the Final Floor.

Full Source Code

The complete, unabridged C source code for the "Echoes of the Old Tower" RPG Game is provided below, spanning the following pages.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* -----
   CONSTANTS
   ----- */
#define MAX_NAME      50
#define MAX_LOGIN     40
#define QUIZ_POOL     20 /* how many random-pool questions exist */
#define QUIZ_FIXED     3 /* always-asked questions */
#define QUIZ_RANDOM    4 /* random questions picked from the pool */
#define SAVE_FILE     "story_save.txt"

/* -----
   DATA TYPES
   ----- */

/* The hero. Works for both Free World Mode and Story Mode -- some
   fields (wife_name, pet_name, romance_name) are simply unused
   depending on which mode is active. */
typedef struct {
    char name[MAX_NAME];
    int hp; /* current health */
    int max_hp; /* max health */
    int strength_mult; /* multiplies unarmed-move damage */
    int weapon_mult; /* multiplies weapon damage */

    int has_knife;
    int has_sword;
    int gun_shots;
    int grenades;
    int healing_potions;
    int machine_gun_uses;

    int has_pet; /* 0 none, 1 cat, 2 dog */
    char pet_name[MAX_NAME];

    char wife_name[MAX_NAME]; /* Story Mode only */
    int has_companion; /* Free World romance NPC */
    char companion_name[MAX_NAME];
} Player;

/* A single enemy "template". Combat makes a working copy so the
   template's HP is never permanently changed. */
typedef struct {
    char name[MAX_NAME];
    int hp; /* enemy HP */
    int damage; /* damage dealt to hero per hit */
    int miss_chance; /* 0-100, chance the enemy's hit misses */
    int retaliate_every; /* enemy strikes back after this many
                          LANDED hits from the hero (1 = every
```

Full Source Code (Cont.)

```

                                hit, 2 = every second hit, etc.)    */
} Enemy;

typedef struct {
    char text[220];
    char option1[100];
    char option2[100];
    char option3[100];
    int correct;                /* 1, 2 or 3 */
} QuizQuestion;

/* =====
SECTION 0 -- SMALL INPUT/OUTPUT HELPERS                [shared/CORE]
===== */

void pause_game(void)
{
    char line[16];
    printf("\n(press ENTER to continue) ");
    if (!fgets(line, sizeof(line), stdin)) exit(0);
}

/* Reads one integer in [min,max]. Re-prompts on anything invalid
(letters, out-of-range numbers, empty lines, trailing garbage). */
int read_int(const char *prompt, int min, int max)
{
    char line[64];
    int value;
    char extra;

    while (1) {
        printf("%s", prompt);
        if (!fgets(line, sizeof(line), stdin)) exit(0);

        if (sscanf(line, " %d %c", &value, &extra) == 1 &&
            value >= min && value <= max) {
            return value;
        }
        printf("Please enter a number from %d to %d.\n", min, max);
    }
}

/* Reads a non-empty line of text into dest (spaces allowed). */
void read_line(const char *prompt, char *dest, int size)
{
    char buffer[256];
    while (1) {
        printf("%s", prompt);
        if (!fgets(buffer, sizeof(buffer), stdin)) exit(0);
        buffer[strcspn(buffer, "\n")] = '\0';
        if (strlen(buffer) == 0) {
            printf("Please type something.\n");
            continue;
        }
        strncpy(dest, buffer, size - 1);
        dest[size - 1] = '\0';
        return;
    }
}

/* =====
SECTION 1 -- QUIZ QUESTION BANK                        [QUIZ]
Shared by the Free World S-Class Black Hulk fight and the Story
Mode final boss fight, exactly as the design doc requires: one
array, used from both places.
===== */

/* EDIT QUIZ QUESTIONS
The 3 questions that are ALWAYS asked, in order. */
static QuizQuestion fixed_questions[QUIZ_FIXED] = {
    { "Who got to the moon for the very first time?",
      "Neil Armstrong", "Adnan Kondakar", "Rashed Hasan", 1 },

```

Full Source Code (Cont.)

```
{ "What is the name of the highest mountain in the world?",
  "Aritro the Peak", "Mount Everest", "Tajingdong", 2 },
{ "What is the favourite word of K M M U sir?",
  "Ghorar Dim", "Hunky Punky", "Bondhu", 3 }
/* ^ TODO: nobody but your instructor knows the "real" answer to
   this in-joke question -- change the correct option (1/2/3)
   to whatever it actually is. */
};

/* EDIT_QUIZ_QUESTIONS
   The pool that 4 random questions are drawn from each fight.
   (Every computed answer below was re-checked by hand -- the
   prototype code had 5 of these marked wrong.) */
static QuizQuestion question_pool[QUIZ_POOL] = {
  { "int x = 17 % 5 + 3 * 2; What is x?",          "8", "13", "10", 1 },
  { "int a=10,b=3; printf(\"%d\", a/b); Output?",    "3", "3.33", "4", 1 },
  { "int x=10; x = x % 4 + 2; What is x?",          "4", "6", "8", 1 },
  { "int x=5; if(x>2 && x<10) printf(\"YES\"); else printf(\"NO\"); Output?",
    "YES", "NO", "ERROR", 1 },
  { "Which function compares two strings in C?",    "strcmp()", "strcpy()", "strlen()", 1
},
  { "int x=2; for(i=0;i<4;i++) x=x+3; Final x?",    "11", "14", "15", 2 },
  { "int a=7,b=2; printf(\"%d\", a*b + a/b); Output?", "3", "4", "5", 2 },
  { "Which keyword skips the current loop iteration?", "break", "continue", "skip", 2 },
  { "int x=5; x++; x*=2; printf(\"%d\",x); Output?",    "10", "12", "11", 2 },
  { "int x = 2 + 3 * 4 % 5; What is x?",          "4", "6", "14", 1 },
  { "Who was first to reach the South Pole?",      "Robert Falcon Scott", "Roald
Amundsen", "Ernest Shackleton", 2 },
  { "Largest ocean on Earth?",                    "Atlantic", "Indian", "Pacific", 3 },
  { "Country with the largest land area?",         "Canada", "Russia", "China", 2 },
  { "Who developed general relativity?",           "Newton", "Einstein", "Galileo", 2 },
  { "Smallest planet in the Solar System?",       "Mercury", "Mars", "Venus", 1 },
  { "Capital city of Australia?",                 "Sydney", "Melbourne", "Canberra", 3
},
  { "Element with chemical symbol \"Au\"?",          "Silver", "Gold", "Copper", 2 },
  { "Who wrote the novel \"1984\"?",                "George Orwell", "Aldous Huxley",
"Ernest Hemingway", 1 },
  { "Organ primarily responsible for filtering waste from the blood?",
    "Heart", "Kidney", "Liver", 2 },
  { "Civilization that built Machu Picchu?",       "Maya", "Aztec", "Inca", 3 }
};

/* =====
SECTION 2 -- PLAYER HELPERS                                [CORE-01]
===== */

void heal_player(Player *p, int amount)
{
  p->hp += amount;
  if (p->hp > p->max_hp) p->hp = p->max_hp;
  printf("You recover %d hp. hp: %d/%d\n", amount, p->hp, p->max_hp);
}

void print_player(const Player *p)
{
  printf("\n-----\n");
  printf("%-18s %d/%d hp\n", p->name, p->hp, p->max_hp);
  if (p->has_knife)      printf("Knife\n");
  if (p->has_sword)     printf("Sword\n");
  if (p->gun_shots > 0)  printf("Gun shots left: %d\n", p->gun_shots);
  if (p->grenades > 0)  printf("Grenades: %d\n", p->grenades);
  if (p->machine_gun_uses > 0) printf("Machine gun uses left: %d\n", p->machine_gun_uses);
  if (p->healing_potions > 0) printf("Healing potions: %d\n", p->healing_potions);
  printf("-----\n");
}

/* =====
SECTION 3 -- COMBAT ENGINE                                [CORE-03, shared]
One fight routine used everywhere: Free World random encounters,
Story Mode room fights, and the Dwyen tutorial fight.
The two Black Hulk quiz-fights use quiz_battle() instead (Section 4).
===== */

/* Presents the hero's move list for this turn and returns the damage
dealt (0 if the action wasn't an attack, e.g. healing or viewing
status), or -1 if the action was invalid/unavailable (turn is
```

Full Source Code (Cont.)

```
re-prompted, doesn't cost a turn). allow_unarmed selects between
the Free World move list (punches/kicks + light weapons) and the
Story Mode move list (sword/gun/grenade only, per [ST-04]). */
static int player_turn(Player *p, int allow_unarmed)
{
    int choice, damage;

    printf("\nChoose an action:\n");
    if (allow_unarmed) {
        printf(" 1) Punch in the belly    (5 dmg)\n");
        printf(" 2) Punch in the face      (10 dmg)\n");
        printf(" 3) Kick in the belly              (20 dmg)\n");
        printf(" 4) Knife                        (12 dmg)%s\n", p->has_knife ? "" : " [not carried]");
        printf(" 5) Sword                      (50 dmg)%s\n", p->has_sword ? "" : " [not carried]");
        printf(" 6) Gun                        (55 dmg, %d shots left)\n", p->gun_shots);
        printf(" 7) Grenade                    (100 dmg, %d left)\n", p->grenades);
        printf(" 8) Machine gun                (200 dmg, %d uses left)\n", p->machine_gun_uses);
        printf(" 9) Healing potion             (+50 hp, %d left)\n", p->healing_potions);
        printf("10) View status\n");

        choice = read_int("> ", 1, 10);
        switch (choice) {
            case 1: return 5 * p->strength_mult;
            case 2: return 10 * p->strength_mult;
            case 3: return 20 * p->strength_mult;
            case 4:
                if (!p->has_knife) { printf("You don't have a knife.\n"); return -1; }
                return 12 * p->weapon_mult;
            case 5:
                if (!p->has_sword) { printf("You don't have a sword.\n"); return -1; }
                return 50 * p->weapon_mult;
            case 6:
                if (p->gun_shots <= 0) { printf("Out of gun shots.\n"); return -1; }
                p->gun_shots--;
                return 55 * p->weapon_mult;
            case 7:
                if (p->grenades <= 0) { printf("No grenades left.\n"); return -1; }
                p->grenades--;
                return 100 * p->weapon_mult;
            case 8:
                if (p->machine_gun_uses <= 0) { printf("No machine-gun uses left.\n"); return
-1; }
                p->machine_gun_uses--;
                return 200 * p->weapon_mult;
            case 9:
                if (p->healing_potions <= 0) { printf("No healing potions left.\n"); return -1;
}
                p->healing_potions--;
                heal_player(p, 50);
                return 0;
            default:
                print_player(p);
                return -1;
        }
    }

    /* Story Mode move list [ST-04] */
    /* Story Mode move list [ST-04] */
    printf(" 1) Sword                (%d dmg)%s\n", 70 * p->weapon_mult, p->has_sword ? "" : " [not
carried]");
    printf(" 2) Gun                    (%d dmg, %d shots left)\n", 85 * p->weapon_mult, p->gun_shots);
    printf(" 3) Grenade                  (%d dmg, %d left)\n", 200 * p->weapon_mult, p->grenades);
    printf(" 4) Healing potion           (+100 hp, %d left)\n", p->healing_potions);
    printf(" 5) View status\n");

    choice = read_int("> ", 1, 5);
    switch (choice) {
        case 1:
            if (!p->has_sword) { printf("You don't have a sword.\n"); return -1; }
            damage = 70 * p->weapon_mult;
            break;
        case 2:
            if (p->gun_shots <= 0) { printf("Out of gun shots.\n"); return -1; }
            p->gun_shots--;
            damage = 85 * p->weapon_mult;
```

Full Source Code (Cont.)

```
        break;
    case 3:
        if (p->grenades <= 0) { printf("No grenades left.\n"); return -1; }
        p->grenades--;
        damage = 200 * p->weapon_mult;
        break;
    case 4:
        if (p->healing_potions <= 0) { printf("No healing potions left.\n"); return -1; }
        p->healing_potions--;
        heal_player(p, 100);
        return 0;
    default:
        print_player(p);
        return -1;
    }
    return damage;
}

/* The shared fight loop [CORE-03]. Returns 1 if the hero wins,
0 if the hero dies. enemy_template is passed by value on purpose
-- the copy inside this function is what takes damage, so calling
code can reuse the same Enemy struct for future fights. */
int fight(Player *p, Enemy enemy_template, int allow_unarmed)
{
    Enemy enemy = enemy_template;
    int landed_hits = 0; /* hero hits that actually damaged the enemy,
                           used for "hits back every Nth hit" enemies.
                           (Healing / invalid turns do NOT count --
                           this was a bug in the original prototype.) */

    int result;

    printf("\n===== \n");
    printf("A %s appears! (%d HP)\n", enemy.name, enemy.hp);
    printf("===== \n");

    while (p->hp > 0 && enemy.hp > 0) {
        printf("\nYour hp: %d/%d | %s hp: %d\n", p->hp, p->max_hp, enemy.name, enemy.hp);

        /* --- OUT OF RESOURCES CHECK --- */
        int can_attack = p->has_sword ||
            p->has_knife ||
            (allow_unarmed) ||
            p->gun_shots > 0 ||
            p->grenades > 0 ||
            p->machine_gun_uses > 0 ||
            p->healing_potions > 0;

        if (!can_attack) {
            printf("\n[OUT OF RESOURCES!] You have no ammo, grenades, or usable weapons
left!\n");
            printf("Press Enter to brace for the enemy's attack...");

            while (getchar() != '\n');

            /* Enemy attacks automatically */
            p->hp -= enemy.damage;
            if (p->hp < 0) p->hp = 0;
            printf("\n%s hits you for %d damage! Your hp: %d/%d\n", enemy.name, enemy.damage, p-
>hp, p->max_hp);

            if (p->hp <= 0) break;
            continue; /* Skip turn and loop again */
        }

        result = player_turn(p, allow_unarmed);

        if (result == -1) continue; /* invalid move, try again */
        if (result == 0) continue; /* healed / viewed status */
        enemy.hp -= result;
        if (enemy.hp < 0) enemy.hp = 0;
        printf("You deal %d damage. %s hp: %d\n", result, enemy.name, enemy.hp);
        landed_hits++;

        if (enemy.hp <= 0) break;
    }
}
```

Full Source Code (Cont.)

```
/* enemy retaliation */
{
    int n = enemy.retaliate_every > 0 ? enemy.retaliate_every : 1;
    if (landed_hits % n == 0) {
        if (enemy.miss_chance > 0 && (rand() % 100) < enemy.miss_chance) {
            printf("%s attacks but misses!\n", enemy.name);
        } else {
            p->hp -= enemy.damage;
            if (p->hp < 0) p->hp = 0;
            printf("%s hits you for %d damage! Your hp: %d/%d\n",
                enemy.name, enemy.damage, p->hp, p->max_hp);
        }
    }
}

if (p->hp <= 0) {
    printf("\nYou were defeated by the %s.\n", enemy.name);
    return 0;
}
printf("\nYou defeated the %s!\n", enemy.name);
return 1;
}

/* =====
SECTION 4 -- QUIZ ENGINE                                [CORE-05, QUIZ]
3 fixed + 4 random questions. Correct = hero hits the boss, wrong
= hero gets hit. Reused for both the Free World S-Class Black Hulk
and the Story Mode final boss.

Damage-per-correct-answer is derived from the boss's own max HP so
that a perfect run (7/7 correct) ALWAYS defeats the boss, no
matter how much HP it has. (The prototype used a flat 125 damage,
which made both Black Hulk fights mathematically unwinnable even
with a perfect score -- that bug is fixed here.)
===== */

static void ask_one_question(QuizQuestion *q, Player *p, Enemy *boss, int per_correct_damage)
{
    printf("\n%s\n", q->text);
    printf(" 1) %s\n 2) %s\n 3) %s\n", q->option1, q->option2, q->option3);

    if (read_int("Answer: ", 1, 3) == q->correct) {
        boss->hp -= per_correct_damage;
        if (boss->hp < 0) boss->hp = 0;
        printf("Correct! You hit the %s for %d damage. Boss hp: %d\n",
            boss->name, per_correct_damage, boss->hp);
    } else {
        p->hp -= boss->damage;
        if (p->hp < 0) p->hp = 0;
        printf("Wrong! The %s hits you for %d damage. Your hp: %d/%d\n",
            boss->name, boss->damage, p->hp, p->max_hp);
    }
}

/* Picks QUIZ_RANDOM distinct indices out of question_pool. */
static void pick_random_questions(int chosen[QUIZ_RANDOM])
{
    int i, j, candidate, already_used;
    for (i = 0; i < QUIZ_RANDOM; i++) {
        do {
            candidate = rand() % QUIZ_POOL;
            already_used = 0;
            for (j = 0; j < i; j++)
                if (chosen[j] == candidate) already_used = 1;
        } while (already_used);
        chosen[i] = candidate;
    }
}

/* Returns 1 if the hero defeats the boss, 0 if the hero dies. */
```

Full Source Code (Cont.)

```
int quiz_battle(Player *p, Enemy boss_template)
{
    Enemy boss = boss_template;
    int per_correct_damage = (boss.hp + (QUIZ_FIXED + QUIZ_RANDOM - 1)) / (QUIZ_FIXED + QUIZ_RANDOM);
    int chosen[QUIZ_RANDOM];
    int i;

    printf("\n=====\\n");
    printf("QUIZ BATTLE: %s (%d HP)\\n", boss.name, boss.hp);
    printf("%d fixed questions + %d random questions.\\n", QUIZ_FIXED, QUIZ_RANDOM);
    printf("=====\\n");

    pick_random_questions(chosen);

    for (i = 0; i < QUIZ_FIXED && p->hp > 0 && boss.hp > 0; i++)
        ask_one_question(&fixed_questions[i], p, &boss, per_correct_damage);

    for (i = 0; i < QUIZ_RANDOM && p->hp > 0 && boss.hp > 0; i++)
        ask_one_question(&question_pool[chosen[i]], p, &boss, per_correct_damage);

    if (boss.hp <= 0) {
        printf("\\nThe %s is defeated!\\n", boss.name);
        return 1;
    }
    printf("\\nThe %s defeated you.\\n", boss.name);
    p->hp = 0;
    return 0;
}

/* =====
SECTION 5 -- FREE WORLD MODE [FW]
No save/load. Restarts fully from the beginning on death.
===== */

/* EDIT_ENEMY_TABLE_FW */
static Enemy fw_enemy(int tier /* 1=B, 2=A, 3=S */, int index)
{
    static Enemy b_class[3] = {
        { "Goon", 100, 9, 0, 1 },
        { "Thief", 80, 6, 0, 1 },
        { "Wild Dog", 90, 19, 40, 1 },
    };
    static Enemy a_class[3] = {
        { "Robot", 250, 40, 0, 1 },
        { "Cyborg", 400, 60, 20, 1 },
        { "Mutant Dinosaur", 600, 120, 40, 1 },
    };
    static Enemy s_class[1] = {
        { "Black Hulk", 1000, 125, 20, 1 },
    };
    if (tier == 1) return b_class[index];
    if (tier == 2) return a_class[index];
    return s_class[index];
}

/* EDIT_ITEM_TABLE */
static void fw_item_pickup(Player *p)
{
    int choice;
    printf("\\nYou spot a box of items on the ground. You can take only one... What will you take?\\n");
    printf("1) Stick\\n 2) Sword\\n 3) Knife\\n 4) Gun (20 shots)\\n");
    printf("5) Torch\\n 6) Healing potion\\n 7) Grenade\\n 8) Machine gun (2 uses)\\n 9) Leave it\\n");
    choice = read_int("> ", 1, 9);
    switch (choice) {
        case 1: printf("You pick up a stick. (just flavor, no effect)\\n"); break;
        case 2: p->has_sword = 1; printf("Sword acquired!\\n"); break;
        case 3: p->has_knife = 1; printf("Knife acquired!\\n"); break;
        case 4: p->gun_shots += 20; printf("Gun acquired -- 20 shots loaded.\\n"); break;
        case 5: printf("You pick up a torch. (just flavor, no effect)\\n"); break;
        case 6: p->healing_potions++; printf("Healing potion acquired.\\n"); break;
        case 7: p->grenades++; printf("Grenade acquired.\\n"); break;
        case 8: p->machine_gun_uses += 2; printf("Machine gun acquired -- 2 uses loaded.\\n"); break;
    }
    break;
}
```


Full Source Code (Cont.)

```
        default: printf("You leave it behind.\n"); break;
    }
}

static void fw_pet_event(Player *p)
{
    p->has_pet = 1 + rand() % 2;
    printf("\nA cute %s approaches and won't leave your side. It wants to come with you\n", p-
>has_pet == 1 ? "cat" : "dog");
    read_line("Give a name to this cute one: ", p->pet_name, MAX_NAME); /* EDIT_NAMES */
    printf("%s is now traveling with you.\n", p->pet_name);
}

static void fw_romance_event(Player *p)
{
    printf("\nAmid the ruins, someone catches your eye -- and she is none other than your love,
who is fighting to survive just like you..\n");
    read_line("Something inside you , all of a sudden , told you to confess your love towards
her.. And you did it..And guess what , she blushed and said 'It took you that long, you
morron'.. Now she is with you, what was her name again?? : ", p->companion_name, MAX_NAME); /*
EDIT_NAMES */
    p->has_companion = 1;
    printf("%s now roams the city with you. And you two make a wonderful couple\n", p-
>companion_name);
}

/* EDIT_LOVER_DIALOGUES
Helper function to handle random dialogues after meeting his love */
static void fw_lover_dialogue(const Player *p)
{
    /* Array of 4 sample dialogue options */
    static const char *dialogues[4] = {
        "\Are you always this slow, or are you just admiring the view?\n" she teases with a
smirk. You feel kinda teased", /* Teasing */
        "\If we get eaten by a mutant, I'm never talking to you again!\n" she mutters, shaking
her head. You just keep on listening to her", /* Funny */
        "She glances away, softly saying, \"...I'm actually really glad I found you out here,
you know?\n", /* Shy */
        "\Hold still for a second,\n" she says gently, brushing dust off your shoulder. \nStay
safe, okay?\n" /* Caring */
    };

    int index = rand() % 4; /* Pick a random dialogue sequence */
    printf("\n[ %s ]: %s\n", p->companion_name, dialogues[index]);
}

void run_free_world(void)
{
    Player p;
    int b_cleared = 0, a_cleared = 0;
    int distance_traveled = 0;
    int has_met_love = 0; /* Tracks romance so it happens ONLY ONCE */
    int current_zone = 0; /* 0 = Road, 1 = Desert */

    memset(&p, 0, sizeof(p));
    p.max_hp = 400;
    p.hp = 400;
    p.strength_mult = 1;
    p.weapon_mult = 1;

    printf("\n===== \n");
    printf("FREE WORLD MODE\n Here you roam and see the ruins of the existing world\n");
    printf("===== \n");
    read_line("Enter your hero's name: ", p.name, MAX_NAME);
    printf("Starting hp: 400\n");

    while (p.hp > 0) {
        /* Transition to Desert after traveling on the road for 8 steps */
        if (distance_traveled >= 8 && current_zone == 0) {
            current_zone = 1;
            printf("\n===== \n");
            printf("The paved roads fade away into vast, scorching DESERT...\n");
            printf("===== \n");
        }

        /* --- ZONE: ROAD --- */
        if (current_zone == 0) {
            printf("\nYou are traveling along the ruined road. (Distance: %d)\n",
distance_traveled);
            printf("1) Roam the road\n");
            printf("2) Enter nearby building\n");
            printf("3) Quit to main menu\n");
        }
    }
}
```

Full Source Code (Cont.)

```
int choice = read_int("> ", 1, 3);
if (choice == 3) {
    printf("You rest for now.\n");
    return;
}

if (choice == 1) {
    /* ROAMING THE ROAD */
    distance_traveled++;
    printf("\nYou roam further down the highway...\n");

    /* 1. Chance for One-Time Romance Event */
    if (!has_met_love && (rand() % 100 < 25 || distance_traveled >= 5)) {
        fw_romance_event(&p);
        has_met_love = 1;
        continue;
    }

    /* 2. Random Dialogue with Lover (ONLY IF ALREADY MET) */
    if (has_met_love && rand() % 100 < 40) {
        fw_lover_dialogue(&p);
    }

    /* 3. Item Pickup */
    if (rand() % 100 < 30) fw_item_pickup(&p);

    /* 4. Pet Event */
    if (!p.has_pet && rand() % 100 < 25) fw_pet_event(&p);

    /* 5. Enemy Encounter */
    if (rand() % 100 < 50) {
        if (!b_cleared) {
            int enemy_index = rand() % 3;
            Enemy e = fw_enemy(1, enemy_index);
            if (enemy_index == 0) {
                int run_choice = read_int("\nA Goon blocks your way! 1) Fight 2)
Run: ", 1, 2);

                if (run_choice == 2 && rand() % 2 == 0) {
                    printf("You managed to escape safely!\n");
                    continue;
                }
            }
            if (!fight(&p, e, 1)) break;
        } else if (!a_cleared) {
            Enemy e = fw_enemy(2, rand() % 3);
            if (!fight(&p, e, 1)) break;
        } else {
            printf("\nAn S-Class enemy blocks the highway: the Black Hulk!\n");
            quiz_battle(&p, fw_enemy(3, 0));
            if (p.hp <= 0) break;
        }
    }
}
else if (choice == 2) {
    /* ENTERING A BUILDING */
    int floors = 1 + rand() % 4;
    printf("\nYou enter an abandoned building with %d floor(s).\n", floors);

    for (int floor = 1; floor <= floors && p.hp > 0; floor++) {
        printf("\n--- Building Floor %d ---\n", floor);

        /* One-Time Romance can happen inside a building too */
        if (!has_met_love && rand() % 100 < 20) {
            fw_romance_event(&p);
            has_met_love = 1;
        }

        /* Random Dialogue inside building (ONLY IF ALREADY MET) */
        if (has_met_love && rand() % 100 < 35) {
            fw_lover_dialogue(&p);
        }

        if (rand() % 100 < 35) fw_item_pickup(&p);
    }
}
```

Full Source Code (Cont.)

```

        /* Building Enemy Encounter */
        if (!b_cleared) {
            Enemy e = fw_enemy(1, rand() % 3);
            if (!fight(&p, e, 1)) break;
        } else if (!a_cleared) {
            Enemy e = fw_enemy(2, rand() % 3);
            if (!fight(&p, e, 1)) break;
        } else {
            printf("\nThe Black Hulk lurks in this building!\n");
            quiz_battle(&p, fw_enemy(3, 0));
            if (p.hp <= 0) break;
        }
        if (p.hp > 0) pause_game();
    }
}

/* --- ZONE: DESERT --- */
else {
    printf("\nYou are in the vast, endless desert.\n");
    printf("1) Roam the desert wasteland\n");
    printf("2) Quit to main menu\n");

    int choice = read_int("> ", 1, 2);
    if (choice == 2) return;

    distance_traveled++;

    /* One-Time Romance Event */
    if (!has_met_love && rand() % 100 < 30) {
        fw_romance_event(&p);
        has_met_love = 1;
        continue;
    }

    /* Random Dialogue in Desert (ONLY IF ALREADY MET) */
    if (has_met_love && rand() % 100 < 40) {
        fw_lover_dialogue(&p);
    }

    /* Desert Events */
    if (rand() % 100 < 25) fw_item_pickup(&p);
    if (!p.has_pet && rand() % 100 < 20) fw_pet_event(&p);

    /* Desert Enemy Battles */
    if (rand() % 100 < 60) {
        if (!b_cleared) {
            Enemy e = fw_enemy(1, rand() % 3);
            if (!fight(&p, e, 1)) break;
        } else if (!a_cleared) {
            Enemy e = fw_enemy(2, rand() % 3);
            if (!fight(&p, e, 1)) break;
        } else {
            printf("\nThe Black Hulk emerges from the desert dust!\n");
            quiz_battle(&p, fw_enemy(3, 0));
            if (p.hp <= 0) break;
        }
    }
}

if (p.hp > 0) pause_game();
}

printf("\n=====");
printf("You have fallen, %. Free World Mode restarts from the beginning.\n", p.name);
printf("=====");
}

/* =====
SECTION 6 -- STORY MODE [ST]
Has save/load and a single checkpoint (the Ground Floor of the
Old Tower). Only a male hero, per the design doc.
===== */

```

Full Source Code (Cont.)

```
/* EDIT_ENEMY_TABLE_ST */
static Enemy st_ground_enemy(int index)
{
    static Enemy table[4] = {
        { "Street Fighter", 80, 7, 0, 1 },
        { "Big Dog", 60, 10, 0, 1 },
        { "Swordsman", 120, 15, 0, 1 },
        { "Robot", 150, 40, 0, 1 },
    };
    return table[index];
}

static Enemy st_floor1_enemy(int index)
{
    static Enemy table[3] = {
        { "Robot", 150, 40, 0, 1 },
        { "Cyborg", 200, 45, 0, 1 },
        { "Mutant Dinosaur", 180, 50, 50, 1 },
    };
    return table[index];
}

static Enemy st_dragon(void) { Enemy e = { "Mutant Dragon", 500, 100, 0, 1 }; return e; }
static Enemy st_mini_hulk(void){ Enemy e = { "Mini Hulk", 1000, 200, 0, 1 }; return e; } /*
hits back every 2nd landed hit */
static Enemy st_final_boss(void){ Enemy e = { "Black Hulk", 2000, 220, 0, 1 }; return e; }
static Enemy st_dwye(void) { Enemy e = { "Dwyen", 70, 5, 0, 1 }; return e; }

/* ---- persistence [CORE-02] ----- */

typedef struct {
    int at_checkpoint; /* 0 = not started, 1 = cleared ground floor */
    int floor1_cleared;
    int floor2_cleared;
    int comeback_done;
    int gun_pickup_available; /* the hidden Floor-1 gun, still there? */
    int potion_pickup_available; /* the Floor-2 potions, still there? */
} StoryProgress;

/* Everything is written one field per line as plain text -- easy to
read back with fgets(), and (unlike the prototype's fscanf("%s"...
approach) this safely supports names/passwords containing spaces. */
static int save_game(const char *user, const char *pass, const Player *p, const StoryProgress
*prog)
{
    FILE *fp = fopen(SAVE_FILE, "w");
    if (!fp) { printf("Could not write save file.\n"); return 0; }

    fprintf(fp, "%s\n%s\n", user, pass);
    fprintf(fp, "%s\n%s\n", p->name, p->wife_name);
    fprintf(fp, "%d %d %d %d\n", p->hp, p->max_hp, p->strength_mult, p->weapon_mult);
    fprintf(fp, "%d %d %d %d %d\n", p->has_knife, p->has_sword, p->gun_shots,
        p->grenades, p->healing_potions);
    fprintf(fp, "%d\n", p->machine_gun_uses);
    fprintf(fp, "%d %d %d %d %d %d\n", prog->at_checkpoint, prog->floor1_cleared,
        prog->floor2_cleared, prog->comeback_done,
        prog->gun_pickup_available, prog->potion_pickup_available);

    fclose(fp);
    return 1;
}

static void read_file_line(FILE *fp, char *dest, int size)
{
    if (!fgets(dest, size, fp)) dest[0] = '\0';
    dest[strlen(dest)] = '\0';
}

/* returns 1 = loaded ok, 0 = no save file, -1 = wrong user/password */
static int load_game(const char *user, const char *pass, Player *p, StoryProgress *prog)
{
    FILE *fp = fopen(SAVE_FILE, "r");
    char stored_user[MAX_LOGIN], stored_pass[MAX_LOGIN];
    char line[64];
```

Full Source Code (Cont.)

```
if (!fp) return 0;

read_file_line(fp, stored_user, MAX_LOGIN);
read_file_line(fp, stored_pass, MAX_LOGIN);
if (strcmp(user, stored_user) != 0 || strcmp(pass, stored_pass) != 0) {
    fclose(fp);
    return -1;
}

read_file_line(fp, p->name, MAX_NAME);
read_file_line(fp, p->wife_name, MAX_NAME);

if (!fgets(line, sizeof(line), fp)) { fclose(fp); return 0; }
sscanf(line, "%d %d %d %d", &p->hp, &p->max_hp, &p->strength_mult, &p->weapon_mult);

if (!fgets(line, sizeof(line), fp)) { fclose(fp); return 0; }
sscanf(line, "%d %d %d %d %d", &p->has_knife, &p->has_sword, &p->gun_shots,
        &p->grenades, &p->healing_potions);

if (!fgets(line, sizeof(line), fp)) { fclose(fp); return 0; }
sscanf(line, "%d", &p->machine_gun_uses);

if (!fgets(line, sizeof(line), fp)) { fclose(fp); return 0; }
sscanf(line, "%d %d %d %d %d", &prog->at_checkpoint, &prog->floor1_cleared,
        &prog->floor2_cleared, &prog->comeback_done,
        &prog->gun_pickup_available, &prog->potion_pickup_available);

fclose(fp);
return 1;
}

/* ---- narrative beats ----- */

/* EDIT_INTRO_DIALOGUE */
static void st_intro(void)
{
    printf("\n=====\\n");
    printf("STORY MODE\\n");
    printf("=====\\n");
    printf("\\nYou are wealthy, successful, and a skilled martial artist.\\nHowever, you sure have
a lot of enemies\\n");
    printf("Tonight you are out to dinner with your lovely wife when a sudden ehpllosion rocks
the street outside.\\n She crumbles from fear\\n You want to go to check, but she stops you... You
assure its ok and you will be back soon.\\n");

    if (read_int("\\n1) What happened next? 2) Skip to the game: ", 1, 2) == 1) {
        printf("\\nYou go anyway... But that is the mistake you will forever remember\\n");
        printf("When you turn back, your wife is gone.\\n");
    } else {
        printf("\\n[Skipped. Your wife has been kidnapped.]\\n");
    }

    printf("\\n\\nThe kidnappers appear tied to enemies from your past business dealings.\\n");
    printf("The police can't help. You decide to find their warehouse yourself.\\n");

    if (read_int("\\n1) What happened next? 2) Skip to the game: ", 1, 2) == 1) {
        printf("\\nYou tell your old friend Detective Holmes and spend a day searching for leads
-- nothing.\\n");
        printf("Back home, a letter is waiting for you.\\n");
    } else {
        printf("\\n[Skipped. A letter from Holmes is waiting for you.]\\n");
    }

    printf("\\n\\nHolmes' letter reads:\\n");
    printf(" \\nI have a lead. It's too dangerous to put in writing -- come see me\\n");
    printf(" in person. And watch your back, they may be following you too.\\n");
    pause_game();
}

/* EDIT_DWYEN_DIALOGUE */
static int st_dwyen_fight(Player *p)
{
    Enemy dwyen = st_dwyen();

    printf("\\n\\nOn the way to Holmes, a man in a dark coat steps out of an alley.\\n");
    printf("\\n\\nName's Dwyen. Mooz sends its regards.\\n");
```

Full Source Code (Cont.)

```
printf("Dwyen attacks before you can react!\n");
p->hp -= 25;
if (p->hp < 0) p->hp = 0;
printf("You take 25 damage. hp: %d/%d\n", p->hp, p->max_hp);

if (p->hp <= 0) return 0;

/* Dwyen is fought bare-handed only: punch (20) / kick (50), no
   weapons or items, per [ST-03]. We reuse the shared fight()
   engine but with a tiny custom move list, since Dwyen's moves
   don't match either of the two standard move lists. */
while (p->hp > 0 && dwyen.hp > 0) {
    int choice, damage;
    printf("\nYour hp: %d | Dwyen hp: %d\n", p->hp, dwyen.hp);
    printf(" 1) Punch (20 dmg)\n 2) Kick (50 dmg)\n");
    choice = read_int("> ", 1, 2);
    damage = (choice == 1) ? 20 : 50;

    dwyen.hp -= damage;
    if (dwyen.hp < 0) dwyen.hp = 0;
    printf("You deal %d damage. Dwyen hp: %d\n", damage, dwyen.hp);

    if (dwyen.hp > 0) {
        p->hp -= dwyen.damage;
        if (p->hp < 0) p->hp = 0;
        printf("Dwyen hits back for %d. Your hp: %d/%d\n", dwyen.damage, p->hp, p->max_hp);
    }
}

if (p->hp <= 0) {
    printf("\nDwyen defeats you before you ever reach Holmes.\n");
    return 0;
}
printf("\nYou defeat Dwyen and continue on to Holmes.\n");
return 1;
}

static void st_loadout(Player *p)
{
    int picked[4] = {0, 0, 0, 0};
    int count = 0, choice;

    printf("\nBefore entering the Old Tower, choose exactly 3 of these 4 [ST-04]:\n");
    while (count < 3) {
        printf("\n 1) Sword                (70 dmg, unlimited uses)%s\n", picked[0] ? " [chosen]"
: "");
        printf(" 2) Gun                    (85 dmg, 14 shots)%s\n", picked[1] ? " [chosen]" : "");
        printf(" 3) 5 grenades                (200 dmg each)%s\n", picked[2] ? " [chosen]" : "");
        printf(" 4) 3 healing potions          (+50 hp each)%s\n", picked[3] ? " [chosen]" : "");
        choice = read_int("Pick one: ", 1, 4);
        if (picked[choice - 1]) { printf("Already picked that one.\n"); continue; }
        picked[choice - 1] = 1;
        count++;
        if (choice == 1) p->has_sword = 1;
        else if (choice == 2) p->gun_shots = 16;
        else if (choice == 3) p->grenades = 5;
        else p->healing_potions = 3;
    }
    printf("\nLoadout locked in.\n");
}

/* ---- The Old Tower [ST-05] ----- */

/* Ground floor: 3 rooms with enemies, then stairs. This is the
   CHECKPOINT -- reaching the stairs here is where a story-mode death
   sends the player back to. */
static int st_ground_floor(Player *p)
{
    int room, choice;
    printf("\n===== \nGROUND FLOOR
(checkpoint)\n===== \n");

    for (room = 1; room <= 3; room++) {
        printf("\nRoom %d: an enemy blocks your way. And you have to beat it to move forward\n",
room);
    }
}
```

Full Source Code (Cont.)

```
if (!fight(p, st_ground_enemy(rand() % 4), 0)) return 0;

if (room == 1) {
    printf("Exits: NORTH , EAST .\n");
    choice = read_int("1) East 2) North : ", 1, 2);
    if (choice == 2) { printf("\nYou reach the stairs up.\n"); return 1; }
} else if (room == 2) {
    printf("Exit: NORTH .\n");
} else {
    printf("Exit: WEST.\n");
}
}
printf("\nYou reach the stairs up.\n");
return 1;
}

/* 1st floor: Player chooses EAST (Room 2) or WEST (Room 3) from Room 1.
Both paths lead to the Stairs. A gun hides in one of the rooms. */
static int st_first_floor(Player *p, int *gun_available)
{
    int choice, second_room, gun_room = 2 + rand() % 2; /* room 2 or room 3 */

    printf("\n=====n1ST
FLOOR\n=====n");

    /* --- ROOM 1 (Entry) --- */
    printf("\nRoom 1 (Entry): an enemy blocks the way. You need to wipe the floor with him to
proceed\n");
    if (!fight(p, st_floor1_enemy(rand() % 3), 0)) return 0;

    printf("\nExits available: EAST or WEST .\n");
    choice = read_int("Where do you want to go? 1) East 2) West: ", 1, 2);
    second_room = (choice == 1) ? 2 : 3;

    /* --- ROOM 2 / ROOM 3 --- */
    printf("\nRoom %d: an enemy blocks the way. You have your hands full today, man...\n",
second_room);
    if (!fight(p, st_floor1_enemy(rand() % 3), 0)) return 0;

    /* Check for hidden gun */
    if (*gun_available && second_room == gun_room) {
        printf("\n!! You spot a gun hidden in the corner! You should not miss the chance to grab
it\n");
        if (read_int("1) Take it 2) Leave it: ", 1, 2) == 1) {
            p->gun_shots += 20;
            *gun_available = 0;
            printf("Gun acquired -- 20 shots loaded.\n");
        }
    }

    /* Move to Stairs Room */
    printf("\nExit NORTH leads to the Stairs Room.\n");
    printf("1) Proceed to Stairs Room\n");
    read_int("> ", 1, 1);

    printf("\nYou reach the stairs up to proceed.\n");
    return 1;
}

/* 2nd floor:
Room 1 (Entry) -> choice to go WEST (Room 2 / Potions) or NORTH (Room 3 / Dragon)
Room 2 (Potions) -> exit NORTH leads to Room 3 (Dragon)
Room 3 (Dragon) -> exit NORTH leads to Stairs Room (Mini Hulk) */
static int st_second_floor(Player *p, int *potion_available, int comeback_done)
{
    int choice;

    printf("\n=====n2ND
FLOOR\n=====n");

    /* --- ROOM 1 (Entry) --- */
    printf("\nRoom 1 (Entry): empty. Exits available: WEST or NORTH .\n");
    choice = read_int("Where do you want to go? 1) West 2) North : ", 1, 2);

    if (choice == 1) {
        /* --- ROOM 2 (Potion Room) --- */
        printf("\nRoom 2: empty, but you spot something on a shelf.\n");
    }
}
```

Full Source Code (Cont.)

```
        if (*potion_available) {
            if (read_int(" Well well...A pair of healing potions sits on a shelf. 1) Take 2)
Leave: ", 1, 2) == 1) {
                p->healing_potions += 2;
                *potion_available = 0;
                printf("You take 2 healing potions.\n");
            }
        } else {
            printf("The shelf is empty.\n");
        }

        printf("\nExit NORTH leads to Room 3.\n");
        printf("1) Proceed to Room 3\n");
        read_int("> ", 1, 1);
    }

    /* --- ROOM 3 (Mutant Dragon) --- */
    printf("\nRoom 3: a Mutant Dragon roars and blocks the way! Damn, thats a big lizard\n");
    if (!fight(p, st_dragon(), 0)) return 0;

    printf("\nExit NORTH leads to the Stairs Room.\n");
    printf("1) Proceed to Stairs Room\n");
    read_int("> ", 1, 1);

    /* --- STAIRS ROOM (Mini Hulk) --- */
    printf("\nYou reach the Stairs Room -- a massive shape rises in the dark. The Mini Hulk! His
name can have mini in it\n But surely he is not mini\n");

    if (!comeback_done) {
        /* Scripted loss on first attempt */
        printf("\nYou trade blows with the Mini Hulk, but it's simply too strong.\n And he beat
the crap out of you\n");
        printf("It hurls you clean out of the Old Tower.\n");
        p->hp = 1;
        return 2;
    }

    /* Second attempt with comeback power-up */
    if (!fight(p, st_mini_hulk(), 0)) return 0;
    printf("\nThe Mini Hulk falls at last.\n");
    return 1;
}

/* EDIT_HOLMES_DIALOGUE */
static void st_comeback(Player *p)
{
    printf("\n===== \nHOLMES FINDS
YOU\n===== \n");
    printf("\nI told you they wouldn't go down easy,.. man,.. they have beaten you up badly...\n"
Holmes says, hauling you up.\n");
    printf("\nDrink this. All of it.Its a potion i got from an old man named Chio, he said that
he was your martial artist master\n");

    p->max_hp = 900;
    p->hp = 900;
    p->gun_shots +=10;
    p->grenades +=3;
    p->strength_mult *= 2;
    p->weapon_mult *= 2;
    p->healing_potions += 3;

    printf("\nThe super potion takes hold -- your strength and every weapon's power DOUBLE. And
you got some extra weopons too\n");
    printf("Max hp is now 900, fully restored. You gain 2 extra healing potions.\n");
    printf("\nYou head back into the Old Tower.\n");
    pause_game();
}

/* EDIT_ENDING_DIALOGUE */
static void st_ending(const Player *p)
{
    printf("\n===== \nFINAL
ROOM\n===== \n");
    printf("Through the last door, you find %s, shaken but alive.\n You grab her and ask if she
is ok or not\n She says in shaken voice,'They kept me here for how long I don;t know..\n plsss
take me away from here, I want to go home'\n Then you hug her tightly and assure her that its
all ok, they wont be able to hurt you anymore\n", p->wife_name);
    printf("You cut her free and carry her out of the Old Tower, back to the life you built
together.\n ");
    printf("\n*** CONGRATULATIONS -- YOU HAVE COMPLETED STORY MODE! ***\n");
}
```


Full Source Code (Cont.)

```
void run_story_mode(void)
{
    Player p;
    StoryProgress prog = {0, 0, 0, 0, 1, 1};
    char user[MAX_LOGIN], pass[MAX_LOGIN];
    int choice, loaded, floor2_result;

    memset(&p, 0, sizeof(p));
    p.max_hp = 250;
    p.hp = 250;
    p.strength_mult = 1;
    p.weapon_mult = 1;

    printf("\n===== \nSTORY MODE
LOGIN\n===== \n");
    printf(" 1) New player\n 2) Returning player\n");
    choice = read_int("> ", 1, 2);

    if (choice == 1) {
        read_line("Choose a user ID: ", user, MAX_LOGIN);
        read_line("Choose a password: ", pass, MAX_LOGIN);
        read_line("Your hero's name: ", p.name, MAX_NAME);          /* EDIT_NAMES */
        read_line("Your wife's name: ", p.wife_name, MAX_NAME);    /* EDIT_NAMES */

        st_intro();
        if (!st_dwyen_fight(&p)) {
            printf("\nGame over. Story Mode restarts from the beginning.\n");
            return;
        }

        printf("\nYou reach Detective Holmes.\n\"She's being held at the Old Tower,\" he says.
\"Gear up.\n\n");
        st_loadout(&p);
        save_game(user, pass, &p, &prog);
    } else {
        read_line("User ID: ", user, MAX_LOGIN);
        read_line("Password: ", pass, MAX_LOGIN);
        loaded = load_game(user, pass, &p, &prog);
        if (loaded == 0) { printf("\nNo save file found. Start a new game first.\n"); return; }
        if (loaded == -1) { printf("\nIncorrect user ID or password.\n"); return; }
        printf("\nWelcome back, %s.\n", p.name);
    }

    /* Ground floor (checkpoint) */
    while (!prog.at_checkpoint) {
        if (!st_ground_floor(&p)) {
            printf("\nYou died before reaching the checkpoint. Story restarts from the
beginning.\n");
            remove(SAVE_FILE);
            return;
        }
        prog.at_checkpoint = 1;
        save_game(user, pass, &p, &prog);
    }

    /* 1st floor */
    while (!prog.floor1_cleared) {
        if (!st_first_floor(&p, &prog.gun_pickup_available)) {
            printf("\nYou died. Returning to the checkpoint (Ground Floor).\n");
            p.hp = p.max_hp;
            p.gun_shots += 5;
            p.grenades += 1;
            save_game(user, pass, &p, &prog);
            continue;
        }
        prog.floor1_cleared = 1;
        save_game(user, pass, &p, &prog);
    }

    /* Combined Floor 1 & Floor 2 loop */
    while (!prog.floor2_cleared) {

        /* 2nd floor */
        floor2_result = st_second_floor(&p, &prog.potion_pickup_available, prog.comeback_done);
```

Full Source Code (Cont.)

```
        if (floor2_result == 0) {
            printf("\nYou died on Floor 2. Returning to Floor 1...\n");
            p.hp = p.max_hp;
            p.gun_shots += 10;
            p.grenades += 3;
            prog.floor1_cleared = 0; /* Reset floor 1 so you must fight through it again */
            save_game(user, pass, &p, &prog);
            continue;
        }

        if (floor2_result == 2) {
            prog.comeback_done = 1;
            st_comeback(&p); /* Gives super potion powers */
            prog.floor1_cleared = 0; /* Sends you back to Floor 1 with super powers! */
            save_game(user, pass, &p, &prog);
            continue;
        }

        prog.floor2_cleared = 1;
        save_game(user, pass, &p, &prog);
    }

/* Final floor: boss room, then the ending. */
    printf("\n=====FINAL FLOOR\n=====");

    if (quiz_battle(&p, st_final_boss())) {
        st_ending(&p);
        remove(SAVE_FILE); /* story complete, clear the save */
        return;
    }

/* If player loses the quiz battle against Black Hulk */
    printf("\nYou lost to the Black Hulk. Returning to the checkpoint (Ground Floor)...\n");

    p.hp = p.max_hp; /* Restore health */
    prog.floor1_cleared = 0; /* Reset story progress back to Ground Floor checkpoint */
    prog.floor2_cleared = 0;

    save_game(user, pass, &p, &prog);
    return; } /* Return to main menu / checkpoint load */ /* Return
to main menu / checkpoint load */ /* <--- ADD THIS CLOSING BRACE HERE TO CLOSE run_story_mode()
*/ /* Return to main menu / checkpoint load */
/* =====
SECTION 7 -- MAIN MENU [CORE-04]
===== */

int main(void)
{
    int choice;
    srand((unsigned int) time(NULL));

    printf("=====\n");
    printf("Echoes of the Old Tower\n");
    printf("=====\n");

    while (1) {
        printf("\nMAIN MENU\n 1) Free World Mode\n 2) Story Mode\n 3) Exit\n");
        choice = read_int("> ", 1, 3);

        if (choice == 1) run_free_world();
        else if (choice == 2) run_story_mode();
        else { printf("\nGoodbye!\n"); break; }
    }
    return 0;
}
```

Conclusion

The R.P.G Game successfully demonstrates how a relatively large console application can be designed using structured programming principles in C. The project combines several concepts into one interactive system, including structures, arrays, pointers, functions, loops, conditions, strings, random numbers and file handling.

The two gameplay modes demonstrate different approaches to game design. Free World Mode focuses on exploration and random events, while Story Mode provides a more controlled sequence with checkpoints, save/load support, multiple floors and a final quiz-based boss encounter.

The use of reusable functions such as **fight()** and **quiz_battle()** reduces code duplication and makes the project easier to understand and maintain. The project also demonstrates practical pointer usage by passing structures to functions and modifying them where required.

Future Improvements

- Add a graphical user interface instead of console-only interaction.
 - Use encrypted credentials or a safer persistence method.
 - Add more maps, characters, enemies and quests.
 - Add experience points, levels and character progression.
 - Introduce sound and visual effects in a graphical version.
 - Add more save slots and separate game profiles.
-

Project Summary

Item	Details
Language	C
Course	Structured Programming (CSE103)
Project	R.P.G Game — Echoes of the Old Tower
Interface	Console / menu driven
Main concepts	Structures, arrays, pointers, functions, loops, strings, randomization and file handling
Major modes	Free World Mode and Story Mode

— End of Report —